



操作系统课程实验

——从0到1的小型内核实现

2025年 秋季学期

lab-8 文件系统之上层抽象

1. inode管理文件的逻辑
2. / ..
3. 文件系统缓存
4. 文件增删改查底层逻辑
5. 总结

1. 深入理解inode

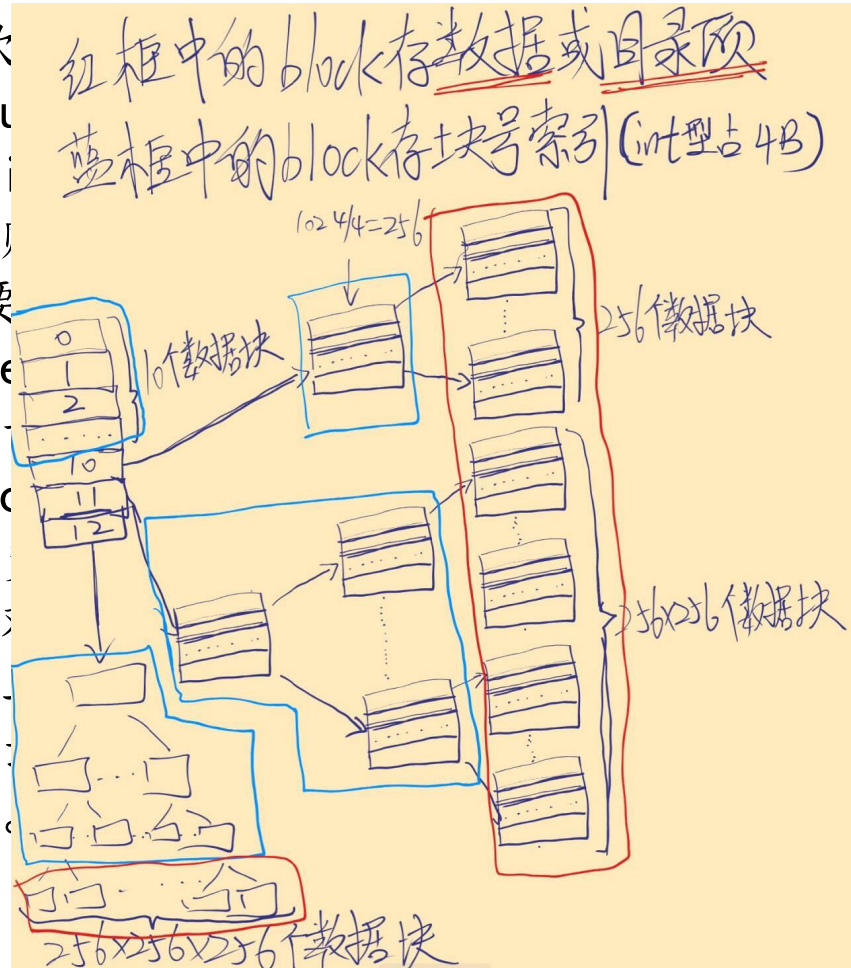
上次实验简单介绍了 **inode_disk** 个字段的作用，没有进一步解释每个字段具体是如何管理不同类型的文件的，以及各字段之间的制约关系。

超级块	inode位图	128块存储inode节点	数据位图	8192块数据块
-----	---------	---------------	------	----------

```
// inode 64 byte
typedef struct inode_disk
{
    short type;
    short major;
    short minor;
    short nlink;
    unsigned int size;
    unsigned int addrs[13];
} inode_disk_t;
```

```
// directory entry 32 byte
typedef struct dirent
{
    unsigned short inode_num;
    char name[30];
} dirent_t;
```

type: 本次文件。unmajor和minor统一都用nlink主要这个inode size: 对类型的inc addr: 最10, 11, 12记录普通数据块记录16项。

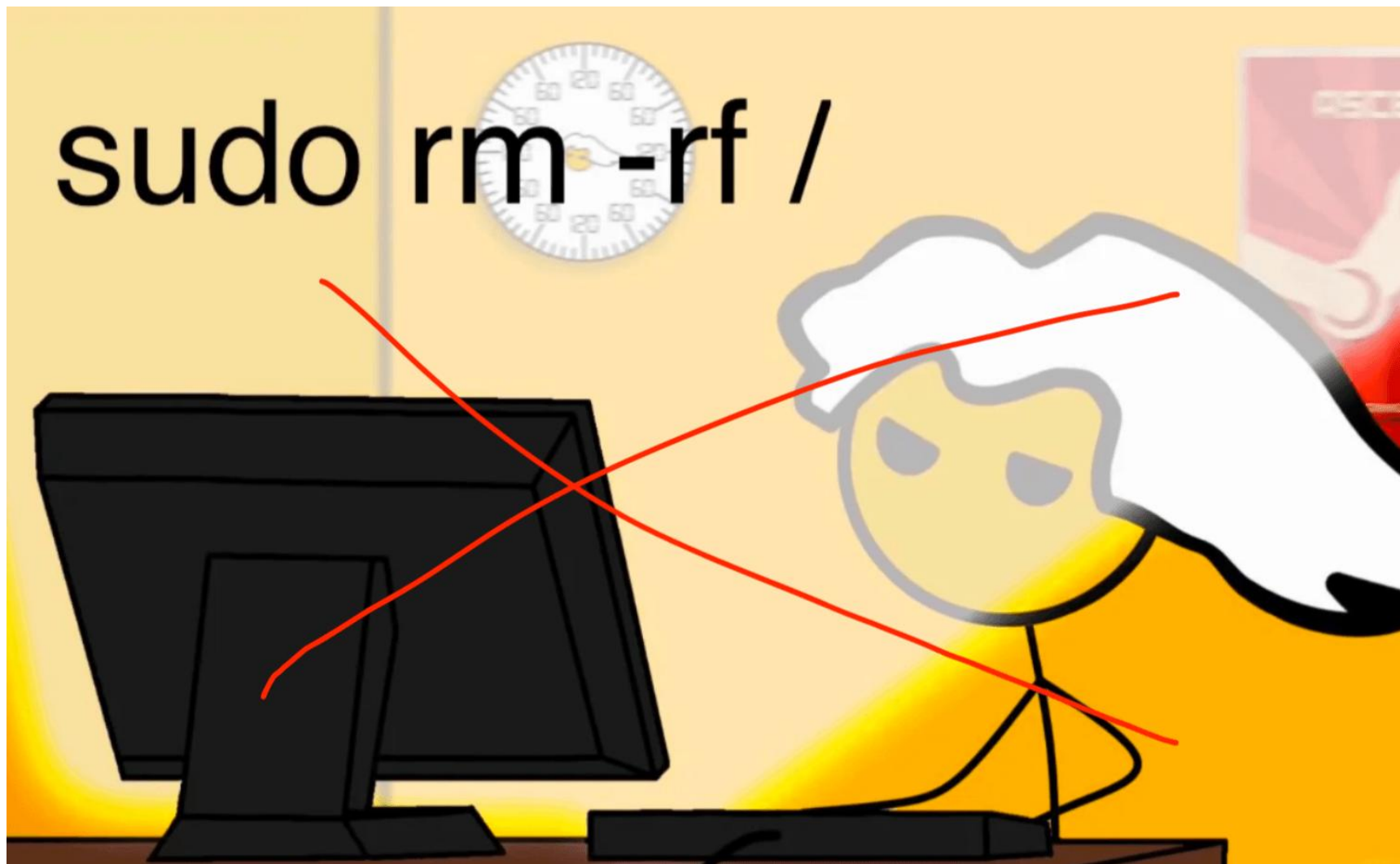


件和普通未来使用。号，本实进程使用于目录色数据块目录来说一个block

2. / ...

linux中“/”就是一切，当然此时的“/”是vfs用户态路径中的“/”，如何理解vfs当中的“/”与硬盘物理分区的关系。

挂载点 = 文件系统
连接到 Linux 目录
树的“接口位置”。



2. / ...

文件系统中的. 和.. 代表目录当前位置和上一级目录位置。

我们在日常经常这样使用：

./helloworld.exe, read("../a/b/c.txt"), cd ..

. 和.. 在目录创建的时候自动创建，在文件系统底层是
目录类型的文件。

在本次实验的mkfs中我们为fs.img创建了初始的"/ ... "，意味着fs.img为根分区。

既然 "/" "." 和 ".." 为目录，那底层对应的inode的type类型就是DIR。

又因为目录中管理的数据为目录项，所以/ inode的目录项有两项分别是：

{name= ".", inode_num=0}, {name= "..", inode_num=0}.

/父目录是自己所以..的inode_num依然是0。

正常情况下.的inode_num为父目录的inodenum, ..的

inode_num为上一级目录的inode_num。

```
// direntory entry 32 byte
typedef struct dirent
{
    unsigned short inode_num;
    char name[30];
} dirent_t;
```

```
(base) xuke@haodong-ThinkStation-P7:~$ mkdir test
(base) xuke@haodong-ThinkStation-P7:~$ cd test
(base) xuke@haodong-ThinkStation-P7:~/test$ la
(base) xuke@haodong-ThinkStation-P7:~/test$ ls -la
```

```
(base) xuke@ThinkStation-P7-number1:~$ file ..
..: directory
```

```
// 准备 . 和 ..
memset(buf, 0, BLOCK_SIZE);
dirent_t *de;
de = (dirent_t *)buf;
de->inode_num = xshort(root_inum);
strcpy(de->name, ".");
de = (dirent_t *) (buf + sizeof(dirent_t));
de->inode_num = xshort(root_inum);
strcpy(de->name, "..");

// 向根目录里写入
unsigned int rooti_block = block_alloc();
block_write(rooti_block, buf);

// 更新rooti
rooti.addrs[0] = xint(rooti_block);
rooti.size = xint(sizeof(dirent_t) * 2);
inode_write(root_inum, &rooti);
```


3. 文件系统缓存

相对于内存来说，磁盘的读写速度是非常慢的。

因此为了减少cpu读写磁盘时候产生的时间开销，需要在内存中建立各种链表或者队列等数据结构来缓存文件系统相关的变量来加速进程对文件的读写。

现代linux中最常见的缓存变量包括：**inode**，**pagecache**（磁盘块缓存数据），**目录项**，**元数据缓存**，**Log Buffer**等。当缓存变量满的时候就会使用各种淘汰策略进行交换。典型的**就是LRU**。它们大部分都是在**内核启动初始化vfs时候建立的**。

本次实验也是借鉴linux思想，建立了两种缓存数据结构：**inode_cache**和**buf_cache**（本质是缓存block）

inode_cache（数组）：主要根据**inode_num**增删改查文件或目录。

buf_cache（双向链表）：主要根据**block_num**读写数据块。

内存中的inode：

```
typedef struct inode
{
    // 磁盘里的inode信息 (由slk保护)
    uint16 type;           // inode 管理的文件类型
    uint16 major;          // 设备文件使用: 主设备号
    uint16 minor;          // 设备文件使用: 次设备号
    uint16 nlink;          // 链接数量 (nlink个文件名链接到这个inode)
    uint32 size;           // 文件大小 (字节)
    uint32 addr[N_ADDRS];  // 文件存储在哪些block里 (分为一级 二级 三级)

    // 内存里的inode信息
    uint16 inode_num;      // inode序号
    uint32 ref;            // 引用数 (由lk_icache保护)
    bool valid;            // 上述磁盘里inode字段的有效性 (由slk保护)
    sleeplock_t slk;       // 睡眠锁
} inode_t;
```

多出的几个字段：

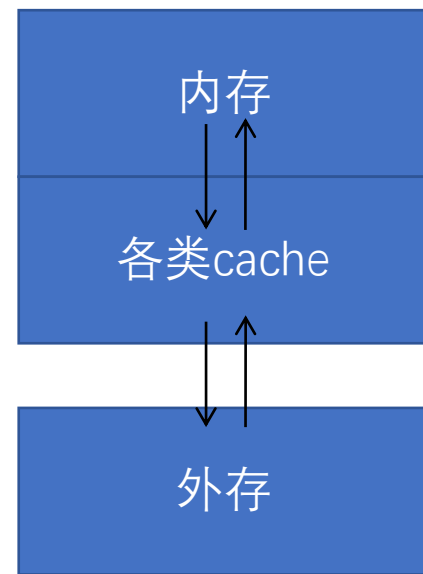
inode_num: 用来索引磁盘块号

ref: **inode**被选中的时候++，被释放的时候--

valid: 判断内存**inode**是否已经从磁盘读出。

slk: 互斥睡眠锁

```
void fs_init()
{
    buf_init(); // 初始化六个块缓冲区
    buf_t *buf; // inode初始化
    inode_init();
}
```



4.文件增删改查底层逻辑

高级编程语言往往通过简单的api来对文件进行增删改查：
fopen()、fread()、fwrite()、fclose()，mkdir命令

```
// 创建磁盘文件
fsfd = open(argv[1], O_RDWR | O_CREAT | O_TRUNC, 0666);
```

fs/method.h中方法的分类：

增：

```
inode_t *inode_create(uint16 type,
                      uint16 major,
                      uint16 minor);
uint32 dir_add_entry(inode_t *pip,
                    uint16 inode_num,
                    char *name);
```

改：

```
uint32 inode_write_data(inode_t *ip,
                       uint32 offset,
                       uint32 len,
                       void *src,
                       bool user);
```

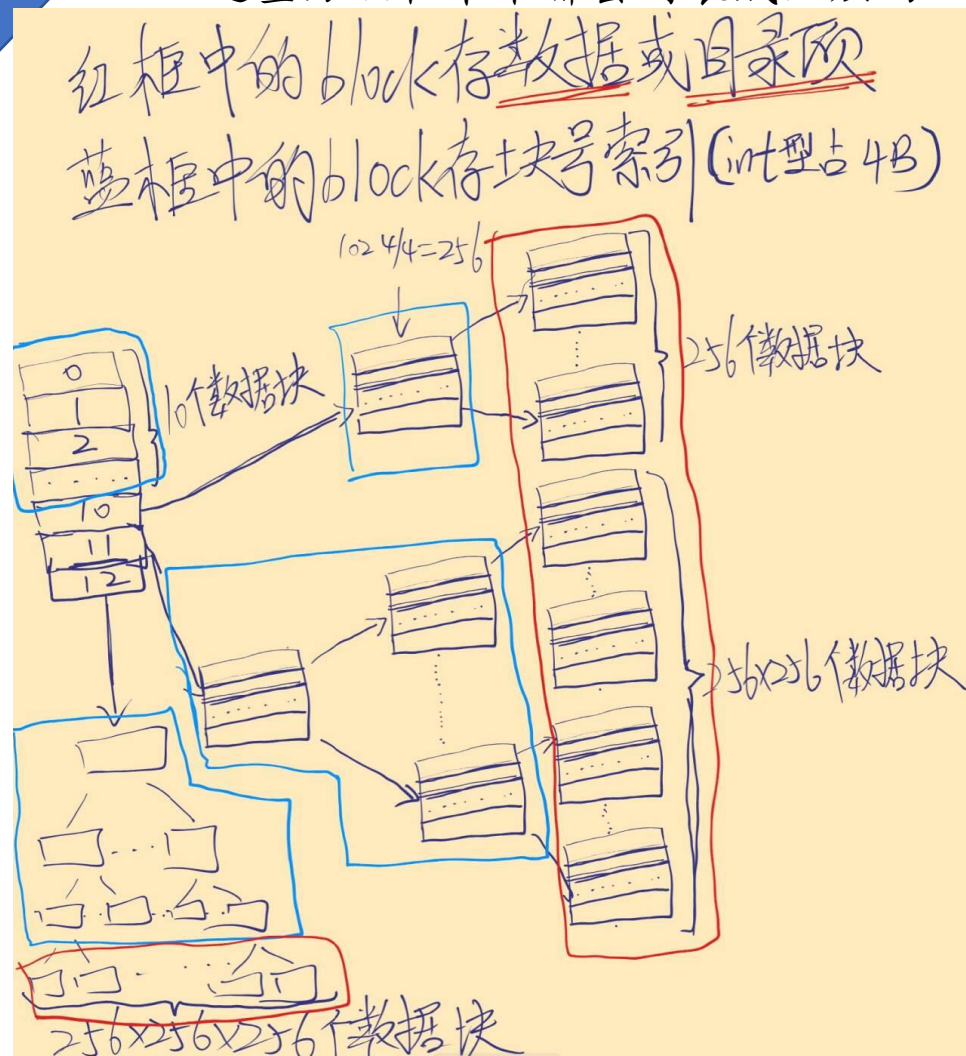
删：

```
void inode_free(inode_t *ip);
uint16 dir_delete_entry(inode_t *pip,
                       char *name);
```

查：

```
uint16 dir_search_entry(inode_t *pip,
                       char *name);
inode_t *path_to_pinode(char *path,
                       char *name);
```

他们最终都是通过系统调用实现，
本次实验就是在实现这些底层方法。
这些方法在未来都会封装成上层的



6. 总结

- 继上一次实验引入文件系统之后，本次实验我们进一步讲解了inode管理普通文件和目录文件的逻辑细节。
- 我们为之前创建的文件系统镜像引入了根inode和..特殊目录。并介绍了“/”在Linux系统的特殊地位。
- 为了加速对磁盘的读写，我们介绍了文件系统的各种缓存数据结构，他们都是基于局部性原理和空间换时间的思想设计的。
- 最后我们介绍了文件系统增删改查的底层inode操作逻辑。他们的实现未来可以在用户态封装成对文件系统的系统调用api。